

PENTESTING AVANZADO

Evasion de Firewalls con nmap

Tecnicas avanzadas de evasion, fragmentacion,
decoys, spoofing y bypass de IDS/IPS

```
-f -mtu -data-length -D -source-port -sS  
-spoof-mac -min-rate -scan-delay -T0 -randomize-hosts
```

Referencia tecnica de ciberseguridad – 2025

Índice

1. Fundamentos: como funcionan los firewalls e IDS	2
1.1. Tipos de inspeccion de paquetes	2
1.2. Que detecta un firewall tipico	2
2. Fragmentacion de paquetes	2
2.1. -f (Fragment packets)	2
2.1.1. Como funciona a nivel de paquetes	2
2.2. -mtu (Establecer MTU personalizado)	3
2.2.1. Estrategia de seleccion de MTU	3
3. Manipulacion del payload	3
3.1. -data-length (Agregar bytes aleatorios al payload)	3
3.1.1. Por que funciona	4
4. Manipulacion de puertos y origen	4
4.1. -source-port (Puerto de origen falsificado)	4
4.1.1. Logica de explotacion	4
4.1.2. Puertos de origen mas efectivos	5
5. Tecnicas de identidad y origen	5
5.1. -D (Decoy scan – IPs senuelo)	5
5.1.1. Diagrama de funcionamiento	5
5.2. -spooof-mac (Falsificar MAC address)	6
5.3. -S (Spoof source IP)	6
6. Tecnicas de timing y velocidad	7
6.1. -T (Timing templates)	7
6.2. -min-rate y -max-rate	7
6.3. -scan-delay y -max-scan-delay	8
7. Tipos de escaneo sigilosos	8
7.1. -sS (SYN scan – Half-open)	8
7.2. -sN, -sF, -sX (NULL, FIN, Xmas scans)	9
7.3. -sA (ACK scan)	9
7.4. -sI (Idle / Zombie scan)	10
7.4.1. Como funciona el Idle Scan	10
8. Tecnicas de orden y estructura	10
8.1. -randomize-hosts	10
8.2. -Pn (No ping / Skip host discovery)	11
8.3. -ip-options (Opciones IP personalizadas)	11
9. Combinaciones avanzadas de evasion	12
9.1. El kit completo: combinando todas las tecnicas	12
9.1.1. Escenario 1: Firewall simple con filtrado de puertos	12
9.1.2. Escenario 2: IDS/IPS con deteccion de escaneos	12
9.1.3. Escenario 3: Firewall estricto en red corporativa	12

9.1.4. Escenario 4: Maxima evasion – objetivo de alto valor	12
10.Tecnicas adicionales avanzadas	13
10.1. –badsum (Checksums incorrectos)	13
10.2. –ttl (TTL personalizado)	13
10.3. –proxies (Cadena de proxies SOCKS/HTTP)	14
10.4. proxychains + nmap	14
10.5. –script (Scripts NSE para evasion)	14
11.Resumen y tabla de referencia rapida	15

1. Fundamentos: como funcionan los firewalls e IDS

Antes de entender las tecnicas de evasion hay que entender lo que se esta evadiendo.

1.1. Tipos de inspeccion de paquetes

Tipo	Capa OSI	Que inspecciona
Packet Filter	3-4	IP origen/destino, puerto, protocolo
Stateful Inspection	3-4	Estado de conexiones TCP (SYN, ACK, etc.)
DPI (Deep Packet)	3-7	Contenido del payload, firmas de ataques
IDS/IPS	3-7	Patrones de comportamiento, anomalias
WAF	7	Payloads HTTP, SQL, XSS

1.2. Que detecta un firewall tipico

- **Port scans:** multiples conexiones SYN a distintos puertos en corto tiempo.
- **SYN floods:** miles de SYN sin completar el handshake.
- **Paquetes malformados:** flags TCP invalidos, tamanos anomalos.
- **Fuentes conocidas de atacantes:** IPs en listas negras (Shodan, Censys).
- **Patrones de escaneo:** velocidad, secuencialidad, TTL anomalo.

Principio de evasion: hacer que el trafico malicioso parezca trafico legitimo, o explotar diferencias en como el firewall y el objetivo interpretan los mismos paquetes.

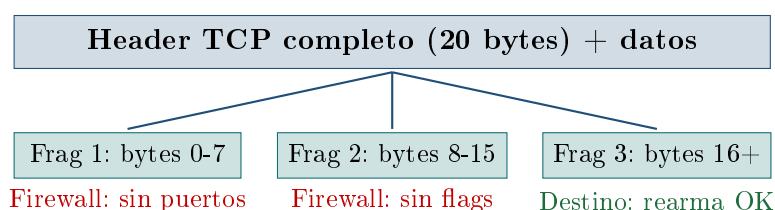
2. Fragmentacion de paquetes

2.1. -f (Fragment packets)

Que hace: divide el header TCP/UDP en fragmentos IP de 8 bytes cada uno. Muchos firewalls y IDS no rearman los fragmentos antes de inspeccionarlos, por lo que no pueden analizar el header completo y dejan pasar los paquetes.

2.1.1. Como funciona a nivel de paquetes

Un header TCP normal tiene 20 bytes. Con -f, nmap lo divide:



Listing 1: Uso de -f

```
1 # Fragmentar en 8 bytes
2 sudo nmap -f 192.168.1.1
3
4 # Doble fragmentacion (fragmentos de 16 bytes, -f -f)
5 sudo nmap -f -f 192.168.1.1
6
7 # Combinado con SYN scan
8 sudo nmap -sS -f -p 80,443,22 192.168.1.1
```

Limitacion: firewalls modernos con *fragment reassembly* rearman los paquetes antes de inspeccionarlos. Efectivo contra firewalls simples o sistemas legacy.

2.2. -mtu (Establecer MTU personalizado)

Que hace: permite especificar el tamaño exacto de cada fragmento IP en múltiplos de 8. Ofrece más control que -f y permite ajustar el tamaño según el firewall objetivo.

Listing 2: Uso de -mtu

```
1 # Fragmentos de 16 bytes
2 sudo nmap --mtu 16 192.168.1.1
3
4 # Fragmentos de 24 bytes
5 sudo nmap --mtu 24 192.168.1.1
6
7 # Fragmentos muy pequenos (maxima confusion)
8 sudo nmap --mtu 8 192.168.1.1
9
10 # IMPORTANTE: debe ser multiplo de 8
11 # sudo nmap --mtu 10 <-- ERROR
```

2.2.1. Estrategia de seleccion de MTU

- **MTU 8:** máxima fragmentación, más lento, más evasivo.
- **MTU 16:** balance entre evasión y velocidad.
- **MTU 1400:** cerca del MTU estándar (1500), evita detección por tamaño anómalo.
- **MTU 24/32:** útil contra IDS con umbrales de detección de fragmentación.

3. Manipulacion del payload

3.1. -data-length (Agregar bytes aleatorios al payload)

Que hace: agrega bytes aleatorios al final de cada paquete enviado, aumentando su tamaño. Muchos IDS tienen firmas basadas en el tamaño exacto de paquetes de escaneo.

nmap por defecto envia paquetes minimos y facilmente detectables por tamano.

3.1.1. Por que funciona

Un SYN de nmap sin opciones tiene exactamente 44 bytes. Un SYN de un navegador real tiene entre 60 y 120 bytes con opciones TCP. Un IDS puede detectar el SYN de 44 bytes como escaneo. Con `-data-length` se simula trafico mas realista.

Listing 3: Uso de `-data-length`

```
1 # Agregar 25 bytes aleatorios
2 sudo nmap --data-length 25 192.168.1.1
3
4 # Simular tamano de paquete de navegador real (60-80 bytes extra)
5 sudo nmap --data-length 50 -sS 192.168.1.1
6
7 # Combinado con fragmentacion
8 sudo nmap -f --data-length 30 -p 80,443 192.168.1.1
9
10 # Para UDP: rellena el payload de cada datagrama
11 sudo nmap -sU --data-length 20 192.168.1.1
```

Limitacion: los bytes agregados son aleatorios y no corresponden a ningun protocolo de aplicacion. DPI avanzado puede detectarlo. Util principalmente contra firewalls con firmas de tamano fijo.

4. Manipulacion de puertos y origen

4.1. `-source-port` (Puerto de origen falsificado)

Que hace: establece un puerto de origen especifico para todos los paquetes enviados. Muchos firewalls tienen reglas permisivas para ciertos puertos de origen considerados “de confianza”: 53 (DNS), 80 (HTTP), 443 (HTTPS), 20 (FTP data), 67 (DHCP).

4.1.1. Logica de explotacion

Un administrador puede configurar una regla como:

ALLOW UDP src_port=53 (respuestas DNS legitimas)

Si nmap falsifica el puerto de origen como 53, esos paquetes pasan la regla aunque no sean respuestas DNS legitimas.

Listing 4: Uso de `-source-port`

```
1 # Simular que el trafico viene del puerto DNS (53)
2 sudo nmap --source-port 53 192.168.1.1
3
4 # Simular trafico HTTP saliente
5 sudo nmap --source-port 80 -sS -p 1-1000 192.168.1.1
6
7 # Puerto 443 (HTTPS) -- muy permisivo en la mayoria de firewalls
```

```

8 sudo nmap --source-port 443 192.168.1.1
9
10 # FTP data port -- a veces en whitelist
11 sudo nmap --source-port 20 192.168.1.1
12
13 # Alternativa con -g (equivalente a --source-port)
14 sudo nmap -g 53 192.168.1.1

```

4.1.2. Puertos de origen mas efectivos

Puerto	Protocolo	Por que funciona
53	DNS	Reglas que permiten respuestas DNS entrantes
80	HTTP	Trafico web de retorno considerado legitimo
443	HTTPS	Muy permisivo, asociado a trafico seguro
67	DHCP	Servidor DHCP, a veces en whitelist interna
20	FTP	Puerto de datos FTP, reglas legacy
123	NTP	Sincronizacion de tiempo, raramente bloqueado

5. Tecnicas de identidad y origen

5.1. -D (Decoy scan – IPs senuelo)

Que hace: envia los paquetes de escaneo mezclados con paquetes que llevan IPs de origen falsificadas (decoys). El firewall/IDS ve el escaneo llegando desde multiples IPs simultaneamente, dificultando identificar cual es el atacante real.

5.1.1. Diagrama de funcionamiento



Listing 5: Uso de -D

```

1 # Decoys manuales con tu IP real mezclada (ME)
2 sudo nmap -D 8.8.8.8,1.1.1.1,ME 192.168.1.1
3
4 # Decoys aleatorios generados por nmap (RND:N = N IPs aleatorias)
5 sudo nmap -D RND:10 192.168.1.1
6
7 # Decoys + SYN scan + puertos especificos
8 sudo nmap -sS -D RND:15 -p 22,80,443 192.168.1.1
9
10 # Decoys con IPs de la misma subred (mas creible)
11 sudo nmap -D 192.168.1.10,192.168.1.20,192.168.1.30,ME 192.168.1.1

```

```
12
13 # Combinar decoys con source port
14 sudo nmap -D RND:5 --source-port 53 -sS 192.168.1.1
```

Advertencia critica: las IPs decoy deben estar activas en la red o el objetivo podria enviar RST a esas IPs inexistentes, lo que puede delatar el escaneo. Usar RND genera IPs completamente aleatorias que pueden no ser enrutables. Lo mas efectivo es usar IPs reales de la misma subred.

5.2. -spoof-mac (Falsificar MAC address)

Que hace: cambia la direccion MAC de origen de los paquetes enviados. Util para evadir controles de acceso basados en MAC (MAC filtering), o para hacer que el trafico parezca provenir de un fabricante especifico.

Listing 6: Uso de -spoof-mac

```
1 # MAC completamente aleatoria
2 sudo nmap --spoof-mac 0 192.168.1.1
3
4 # MAC especifica
5 sudo nmap --spoof-mac 00:11:22:33:44:55 192.168.1.1
6
7 # Simular ser un dispositivo Apple (OUI de Apple)
8 sudo nmap --spoof-mac Apple 192.168.1.1
9
10 # Simular ser un Cisco (routers/switches tipicos en infraestructura)
11 sudo nmap --spoof-mac Cisco 192.168.1.1
12
13 # Simular ser un dispositivo Dell
14 sudo nmap --spoof-mac Dell 192.168.1.1
15
16 # Combinado con decoys
17 sudo nmap --spoof-mac 0 -D RND:5 192.168.1.1
```

Alcance limitado: la MAC solo es visible dentro de la misma red local (capa 2). No sobrevive al primer salto de router. Util en redes LAN donde el firewall filtra por MAC, o en entornos con NAC (Network Access Control) basado en MAC.

5.3. -S (Spoof source IP)

Que hace: falsifica completamente la IP de origen del escaneo. A diferencia de -D, aqui **todos** los paquetes usan la IP falsa. El problema: las respuestas van a la IP falsificada, no al atacante. Solo util en combinacion con -e para captura pasiva o en ataques ciegos.

Listing 7: Uso de -S (IP spoofing total)

```
1 # Falsificar IP completamente
```



```

2 sudo nmap -S 10.0.0.5 -e eth0 192.168.1.1
3
4 # Util para TCP SYN scan ciego (no recibe respuestas)
5 sudo nmap -sS -S 10.0.0.5 -e eth0 -Pn 192.168.1.1
6
7 # NOTA: necesita -e para especificar interfaz de salida
8 # y -Pn porque no puede recibir respuestas de ping

```

6. Tecnicas de timing y velocidad

6.1. -T (Timing templates)

Que hace: controla la velocidad general del escaneo. Los IDS detectan escaneos por la **densidad** de paquetes en el tiempo. Escaneos lentos se confunden con trafico normal y evaden sistemas de deteccion basados en velocidad.

Flag	Nombre	Delay entre probes	Uso
-T0	Paranoid	5 minutos	Maxima evasion, extremadamente lento
-T1	Sneaky	15 segundos	Evasion de IDS, muy lento
-T2	Polite	0.4 segundos	Evasion moderada
-T3	Normal	por defecto	Uso normal
-T4	Aggressive	muy rapido	Redes rapidas, sin evasion
-T5	Insane	sin delay	Maximo ruido, detectable

Listing 8: Uso de templates de timing

```

1 # T0: un probe cada 5 minutos -- evasion maxima
2 sudo nmap -T0 -sS -p 22,80,443 192.168.1.1
3
4 # T1: escaneo lento pero practico
5 sudo nmap -T1 -sS 192.168.1.1
6
7 # Combinado con tecnicas de fragmentacion
8 sudo nmap -T1 -f --source-port 53 192.168.1.1

```

6.2. --min-rate y --max-rate

Que hace: controla exactamente cuantos paquetes por segundo envia nmap, independientemente del template -T. Permite precision quirurgica en la velocidad.

Listing 9: Control preciso de velocidad

```

1 # Minimo 100 paquetes/segundo (rapido para CTFs)
2 sudo nmap --min-rate 100 192.168.1.0/24
3

```

```
4 # Maximo 10 paquetes/segundo (evasión moderada)
5 sudo nmap --max-rate 10 -sS 192.168.1.1
6
7 # Maximo 1 paquete/segundo (muy sigiloso)
8 sudo nmap --max-rate 1 -sS -p 1-1000 192.168.1.1
9
10 # Combinación para escaneo controlado
11 sudo nmap --min-rate 50 --max-rate 100 -sS 192.168.1.0/24
```

6.3. `--scan-delay` y `--max-scan-delay`

Que hace: agrega un delay fijo entre cada probe enviado. Mas granular que `-T`, permite definir tiempos exactos en milisegundos o segundos.

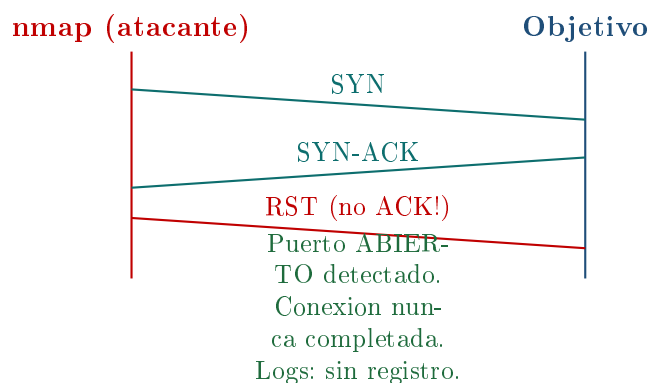
Listing 10: Delays entre probes

```
1 # Esperar 1 segundo entre cada probe
2 sudo nmap --scan-delay 1s -sS 192.168.1.1
3
4 # Delay de 500 milisegundos
5 sudo nmap --scan-delay 500ms 192.168.1.1
6
7 # Delay aleatorio para evitar patrones regulares
8 sudo nmap --scan-delay 200ms --max-scan-delay 2s 192.168.1.1
9
10 # Para IDS con ventanas de detección de 60 segundos:
11 # delay de 70s para caer fuera de la ventana
12 sudo nmap --scan-delay 70s -p 22,80,443 192.168.1.1
```

7. Tipos de escaneo sigilosos

7.1. `-sS` (SYN scan – Half-open)

Que hace: envia un SYN, espera SYN-ACK, pero **nunca completa el handshake**. En lugar de responder con ACK, envia un RST. La conexión nunca se establece completamente, por lo que muchos sistemas de log no la registran.



Listing 11: SYN scan sigiloso

```

1 # SYN scan basico (requiere root)
2 sudo nmap -sS 192.168.1.1
3
4 # SYN scan con todas las tecnicas de evasion
5 sudo nmap -sS -f -D RND:10 --source-port 53 -T1 192.168.1.1
6
7 # SYN scan sin ping previo (no envia ICMP antes de escanear)
8 sudo nmap -sS -Pn 192.168.1.1

```

Diferencia con -sT (Connect scan): -sT completa el handshake completo y luego cierra la conexion. Genera logs en el objetivo. -sS no completa el handshake y muchos sistemas no loggean conexiones incompletas.

7.2. -sN, -sF, -sX (NULL, FIN, Xmas scans)

Que hacen: explotan una ambigüedad en el RFC 793 de TCP. Segun el RFC, si un puerto cerrado recibe un paquete sin SYN/RST/ACK debe responder con RST. Si esta abierto, debe ignorarlo. Los firewalls stateful que solo inspeccionan SYN no bloquean estos paquetes.

Tipo	Flags TCP	Descripcion
-sN (NULL)	Ninguno (0x00)	Sin flags – invalido segun RFC
-sF (FIN)	FIN (0x01)	Solo FIN – sin conexion previa
-sX (Xmas)	FIN + PSH + URG (0x29)	“Arbol de navidad” de flags

Listing 12: Escaneos con flags inusuales

```

1 # NULL scan (sin flags)
2 sudo nmap -sN 192.168.1.1
3
4 # FIN scan
5 sudo nmap -sF 192.168.1.1
6
7 # Xmas scan (FIN + PSH + URG)
8 sudo nmap -sX 192.168.1.1
9
10 # Combinados con evasion
11 sudo nmap -sF -f --source-port 53 -T1 192.168.1.1

```

Limitacion: Windows y Cisco IOS no siguen el RFC al pie de la letra. Responden con RST a todos estos paquetes sin importar si el puerto esta abierto, haciendo que todos los puertos parezcan cerrados. Funcionan bien en sistemas Unix/Linux.

7.3. -sA (ACK scan)

Que hace: envia paquetes ACK sin haber establecido ninguna conexion previa. No detecta si los puertos estan abiertos o cerrados – detecta si estan **filtrados o no filtrados por el firewall**. Sirve para mapear las reglas del firewall, no para encontrar servicios.

Listing 13: ACK scan para mapear firewall

```

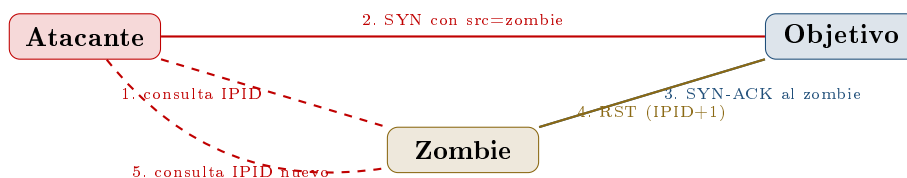
1 # Mapear que puertos filtra el firewall
2 sudo nmap -sA 192.168.1.1
3
4 # Si un puerto responde RST: no filtrado (el paquete llego)
5 # Si no responde nada: filtrado por firewall

```

7.4. -sI (Idle / Zombie scan)

Que hace: el escaneo mas sigiloso posible. Usa un tercer host (“zombie”) para hacer el escaneo. Tu IP real **nunca aparece** en los logs del objetivo. El zombie debe ser un host con trafico IP predecible (IPID incremental).

7.4.1. Como funciona el Idle Scan



Si el IPID del zombie incremento en 2 entre las dos consultas: el puerto del objetivo estaba **abierto**. Si incremento en 1: estaba cerrado.

Listing 14: Idle scan con zombie

```

1 # Primero encontrar un zombie valido (IPID predecible)
2 sudo nmap --script ipidseq 192.168.1.0/24
3
4 # Ejecutar idle scan usando el zombie
5 sudo nmap -sI 192.168.1.50 192.168.1.1
6
7 # Con puerto especifico del zombie
8 sudo nmap -sI 192.168.1.50:80 -p 22,80,443 192.168.1.1

```

El escaneo mas sigiloso existente: el log del objetivo solo muestra la IP del zombie. La IP del atacante real **jamás aparece**. La contramedida es usar sistemas operativos con IPID aleatorio (Linux moderno, Windows Vista+), lo que elimina los zombies validos.

8. Tecnicas de orden y estructura

8.1. --randomize-hosts

Que hace: cuando se escanea un rango de IPs, nmap normalmente las escanea en orden secuencial (192.168.1.1, .2, .3...). Los IDS detectan este patron secuencial como un escaneo de red. `-randomize-hosts` aleatoriza el orden.

Listing 15: Aleatorizacion de hosts

```
1 # Escanear red completa en orden aleatorio
2 sudo nmap --randomize-hosts 192.168.1.0/24
3
4 # Combinado con delay para maxima evasion
5 sudo nmap --randomize-hosts --scan-delay 2s -sS 192.168.1.0/24
6
7 # Con decoys y timing lento
8 sudo nmap --randomize-hosts -T1 -D RND:5 192.168.1.0/24
```

8.2. -Pn (No ping / Skip host discovery)

Que hace: nmap por defecto envia un ping (ICMP Echo) antes de escanear para verificar si el host esta activo. Muchos firewalls bloquean ICMP, haciendo que nmap crea que el host esta caido. `-Pn` salta ese paso y escanea directamente.

Listing 16: Skip host discovery

```
1 # Escanear aunque no responda ping
2 sudo nmap -Pn 192.168.1.1
3
4 # Util para hosts detras de firewall que bloquea ICMP
5 sudo nmap -Pn -sS -p 80,443,22,3389 192.168.1.1
6
7 # Combinacion completa de evasion
8 sudo nmap -Pn -sS -f -D RND:5 --source-port 53 -T1 192.168.1.1
```

8.3. -ip-options (Opciones IP personalizadas)

Que hace: agrega opciones personalizadas al header IP. Algunos firewalls tienen reglas basadas en opciones IP especificas. Tambien permite **IP source routing** (obsoleto pero aun presente en equipos legacy) que fuerza la ruta que sigue el paquete.

Listing 17: Opciones IP personalizadas

```
1 # Loose source routing (LSR) -- forzar ruta
2 sudo nmap --ip-options "L 192.168.1.10 192.168.1.1" target
3
4 # Record route (RR) -- mapear ruta
5 sudo nmap --ip-options "R" target
6
7 # Opciones en hexadecimal
8 sudo nmap --ip-options "\x00\x00\x00\x00" target
```

9. Combinaciones avanzadas de evasión

9.1. El kit completo: combinando todas las tecnicas

En la practica, las tecnicas se combinan para maximizar la evasión. Aqui van estrategias completas segun el escenario:

9.1.1. Escenario 1: Firewall simple con filtrado de puertos

Listing 18: Evasion de firewall simple

```
1 sudo nmap -sS \           # SYN scan (half-open)
2   -f \                   # fragmentacion
3   --source-port 53 \     # puerto origen DNS
4   -Pn \                  # sin ping
5   -p 1-1000 \
6   192.168.1.1
```

9.1.2. Escenario 2: IDS/IPS con deteccion de escaneos

Listing 19: Evasion de IDS/IPS

```
1 sudo nmap -sS \
2   -T1 \                  # timing lento
3   --scan-delay 500ms \   # delay entre probes
4   --randomize-hosts \    # orden aleatorio
5   -D RND:10 \            # 10 decoys
6   --data-length 25 \     # payload aleatorio
7   -f \                   # fragmentacion
8   192.168.1.0/24
```

9.1.3. Escenario 3: Firewall estricto en red corporativa

Listing 20: Evasion corporativa

```
1 sudo nmap -sS \
2   -T0 \                  # paranoid timing
3   -D 10.0.0.5,10.0.0.10,ME \ # decoys internos
4   --source-port 443 \     # origen HTTPS
5   --mtu 16 \              # fragmentos de 16B
6   --data-length 40 \
7   --spoof-mac 0 \         # MAC aleatoria
8   -Pn \
9   --randomize-hosts \
10  192.168.1.1
```

9.1.4. Escenario 4: Maxima evasión – objetivo de alto valor

Listing 21: Evasion maxima

```
1 # Paso 1: encontrar zombie para idle scan
2 sudo nmap --script ipidseq -p 80 192.168.1.0/24
```

```
3
4 # Paso 2: idle scan con zombie (IP real nunca aparece)
5 sudo nmap -sI 192.168.1.50:80 \ # zombie encontrado
6         -sS \
7         -f \
8         -T0 \
9         --scan-delay 2s \
10        -p 22,80,443,3389,8080 \
11        192.168.1.1
```

10. Tecnicas adicionales avanzadas

10.1. `--badsum` (Checksums incorrectos)

Que hace: envia paquetes con checksums TCP/UDP deliberadamente incorrectos. Los hosts reales descartan estos paquetes (checksum incorrecto = paquete corrupto). Si el firewall responde, significa que esta procesando paquetes **sin verificar checksum**, lo que revela su presencia y configuracion. Util para detectar firewalls transparentes.

Listing 22: Deteccion de firewalls con checksum invalido

```
1 # Enviar paquetes con checksum invalido
2 sudo nmap --badsum 192.168.1.1
3
4 # Si hay respuesta: hay un firewall procesando sin validar checksum
5 # Si no hay respuesta: no hay firewall o el checksum se valida
   correctamente
```

10.2. `--ttl` (TTL personalizado)

Que hace: establece un TTL especifico para los paquetes. Util para: (1) confundir fingerprinting del SO basado en TTL, (2) simular que los paquetes vienen de un SO diferente, (3) controlar cuantos saltos puede dar el paquete.

Listing 23: TTL personalizado

```
1 # Simular Windows (TTL tipico: 128)
2 sudo nmap --ttl 128 192.168.1.1
3
4 # Simular Linux (TTL tipico: 64)
5 sudo nmap --ttl 64 192.168.1.1
6
7 # Simular router Cisco (TTL tipico: 255)
8 sudo nmap --ttl 255 192.168.1.1
9
10 # TTL bajo para limitar alcance (no viaja mas de 5 saltos)
11 sudo nmap --ttl 5 192.168.1.1
```

10.3. -proxies (Cadena de proxies SOCKS/HTTP)

Que hace: enruta el trafico de nmap a traves de una cadena de proxies. La IP que ve el objetivo es la del ultimo proxy, no la del atacante real.

Listing 24: Escaneo via proxies

```
1 # Escanear a traves de un proxy SOCKS5
2 sudo nmap --proxies socks5://127.0.0.1:9050 192.168.1.1
3
4 # Usar Tor como proxy SOCKS (anonimato)
5 # Primero iniciar Tor en puerto 9050
6 sudo nmap -sT --proxies socks5://127.0.0.1:9050 -Pn 192.168.1.1
7
8 # Cadena de proxies HTTP
9 sudo nmap --proxies http://proxy1:8080,http://proxy2:8080 192.168.1.1
```

Limitacion critica: -proxies solo funciona con -sT (Connect scan), no con SYN scan. SYN scan requiere paquetes raw que no pueden tunnelizarse por proxies HTTP/SOCKS. Para uso con Tor, considerar proxychains.

10.4. proxychains + nmap

Listing 25: nmap a traves de proxychains

```
1 # Configurar /etc/proxychains.conf
2 # socks5 127.0.0.1 9050 (Tor)
3
4 # Ejecutar nmap via proxychains
5 proxychains nmap -sT -Pn -n 192.168.1.1
6
7 # Con multiples proxies encadenados
8 proxychains nmap -sT -Pn -p 80,443 --open 192.168.1.1
```

10.5. --script (Scripts NSE para evasion)

Listing 26: Scripts NSE utiles para evasion

```
1 # Detectar firewalls y su tipo
2 sudo nmap --script firewall-bypass 192.168.1.1
3
4 # Detectar IDS
5 sudo nmap --script ids-bypass 192.168.1.1
6
7 # Encontrar zombies validos para idle scan
8 sudo nmap --script ipidseq 192.168.1.0/24
9
10 # HTTP con evasion de WAF
11 sudo nmap --script http-waf-detect 192.168.1.1
12 sudo nmap --script http-waf-fingerprint 192.168.1.1
```


11. Resumen y tabla de referencia rapida

Tecnica	Flag	Que evade	Costo
Fragmentacion	-f	Inspeccion de header TCP/UDP	Bajo
MTU custom	-mtu N	Firmas de tamano de paquete	Bajo
Payload extra	-data-length	Firmas de tamano exacto	Bajo
Puerto origen	-source-port	Reglas de firewall por puerto	Bajo
Decoys	-D	Atribucion de origen	Medio
MAC spoofing	-spoof-mac	Filtrado MAC, NAC	Bajo
IP spoofing	-S	Logs del objetivo	Alto
SYN scan	-sS	Logs de conexion completa	Bajo
NULL/FIN/Xmas	-sN/F/X	Firewalls stateful (SYN only)	Bajo
Idle scan	-sI	Atribucion total	Muy alto
Timing lento	-T0/T1	Deteccion por velocidad	Alto
Scan delay	-scan-delay	Ventanas de deteccion IDS	Alto
Aleatorizar hosts	-randomize	Deteccion de patron secuencial	Bajo
Sin ping	-Pn	Firewalls que bloquean ICMP	Nulo
Bad checksum	-badsum	Fingerprinting de firewall	Nulo
TTL custom	-ttl	Fingerprinting de SO por TTL	Bajo
Proxies / Tor	-proxies	Atribucion de IP	Alto

Regla de oro: ninguna tecnica sola es suficiente contra un firewall moderno bien configurado. La evasion efectiva combina **multiples capas**: fragmentacion + timing lento + decoys + puerto de origen confiable + orden aleatorio. Contra IDS/IPS avanzados (Snort, Suricata con reglas actualizadas) el idle scan via zombie es la unica opcion que oculta completamente la IP real del atacante.

Como defenderse: activar fragment reassembly en el firewall, implementar IDS con ventanas de deteccion largas (detecta -T0), usar sistemas con IPID aleatorio (elimina zombies validos), implementar egress filtering (bloquea IP spoofing desde la LAN), y monitorear puertos inusuales de origen (puerto 53 haciendo SYN es anomalo).